

A Brief Explanation of the Implementation of the Optimization-Based Hologram Design Method

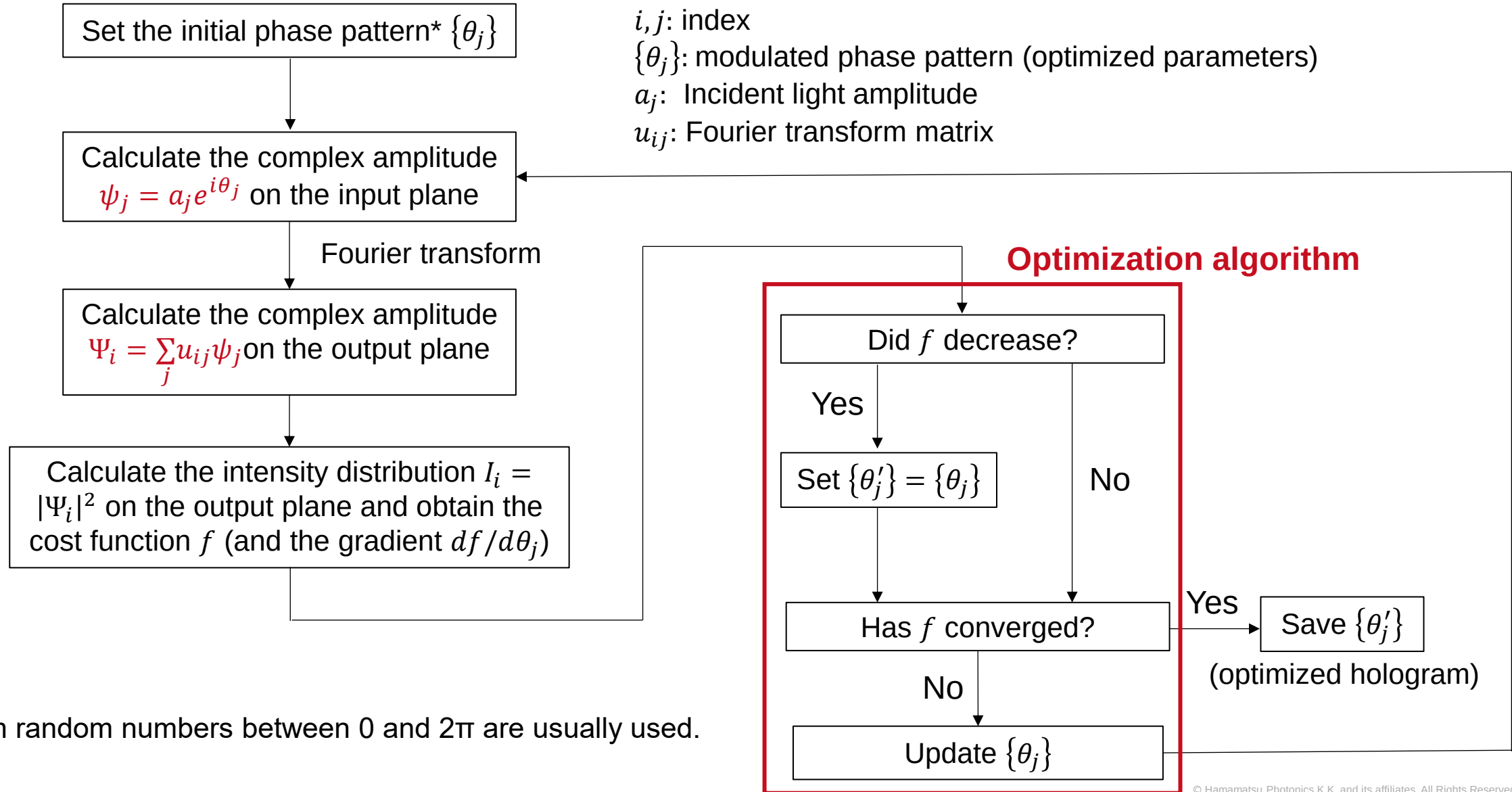
HAMAMATSU PHOTONICS K.K.

1st Group, 1st Research Department, Central Research Laboratory

Keisuke Nishimura

1/29/2026

Flowchart of the Optimization-Based Hologram Design Algorithm



- Among the many optimization algorithms available, we adopted the conjugate gradient method because it offers a good balance of computational cost, convergence speed, and memory usage.
- In many programming languages such as C and Python, there are libraries that allow easy use of multiple optimization algorithms (for example, GSL for C and SciPy for Python).

When performing optimization using such libraries, the only functions you need to implement yourself are:

1. The cost function
2. The gradient of the cost function (for gradient methods)

The desired phase distribution is obtained as an optimized parameters.

Pseudocode for Calculating the Cost Function

$$f_{\text{FOI}} = -\sum_i \tilde{I}_i \tilde{T}_i$$

$$I_i = |\Psi_i|^2, \quad \tilde{I}_i = I_i / N_I, \quad \tilde{T}_i = T_i / N_T,$$

$$N_I = \sqrt{\sum_k I_k^2}, \quad N_T = \sqrt{\sum_k T_k^2}$$

$$i = 0, 1, \dots, N - 1$$

(N : Number of pixels in the computational domain)

Pseudocode

I: imaginary number

```
cost_function(*theta, *a_in, *Ttilde){
    for j {psi[j] = a_in[j]*exp(I*theta[j])}
    Psi = FourierTransform[psi]
    for i {Intensity[i] = |Psi[i]|^2}
    sum = 0
    for i {sum += Intensity[i]^2}
    Norm = sqrt(sum)
    for i {Itilde[i] = Intensity[i]/Norm}
    f = 0
    for i {f += -Itilde[i]*Ttilde[i]}

    return f
}
```

Pseudocode for Calculating the Gradient of the Cost Function

$$\frac{f_{\text{FOI}}}{\partial \theta_j} = \frac{2}{N_I} \sum_i (-\tilde{T}_i + f_{\text{FOI}} \tilde{I}_i) \text{Im}[u_{ij}^* \Psi_j \psi_i^*]$$

Here, we assume the computational domain is a square with side length N_x ($N = N_x^2$).

Pseudocode

Im: imaginary part
conj: complex conjugate

```
gradient(*grad, *theta, *a_in, *Ttilde){  
    (the same code as cost_function in this part.)  
    Set grad to zero array.  
    for jx, jy = 0 to Nx-1 {  
        for ix, iy = 0 to Nx-1 {  
            u = exp[2*π*I*(ix*jx+iy*jy)/Nx]  
            i = iy*Nx + ix  
            j = jy*Nx + jx  
            grad[j] += (2/Norm)*(-Ttilde[i]+f*Itilde[i])*Im(u*Psi[j]*conj(psi[i]))  
        }  
    }  
}
```

www.hamamatsu.com